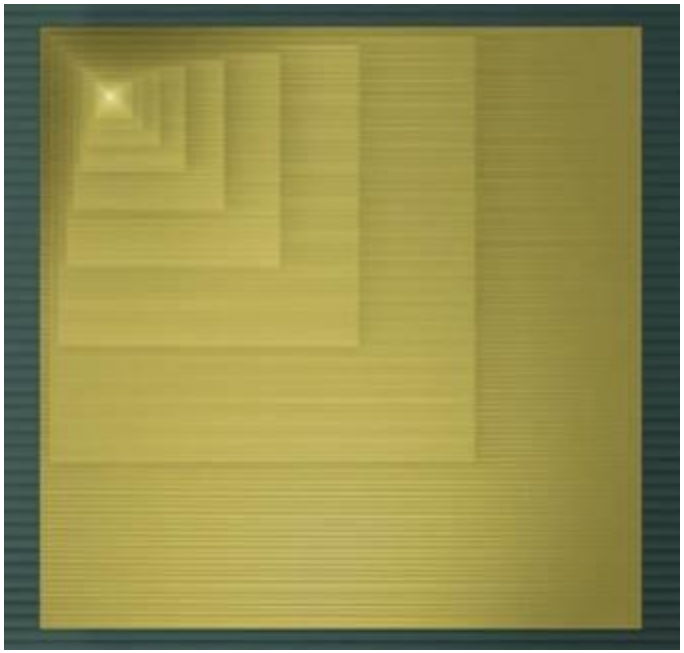
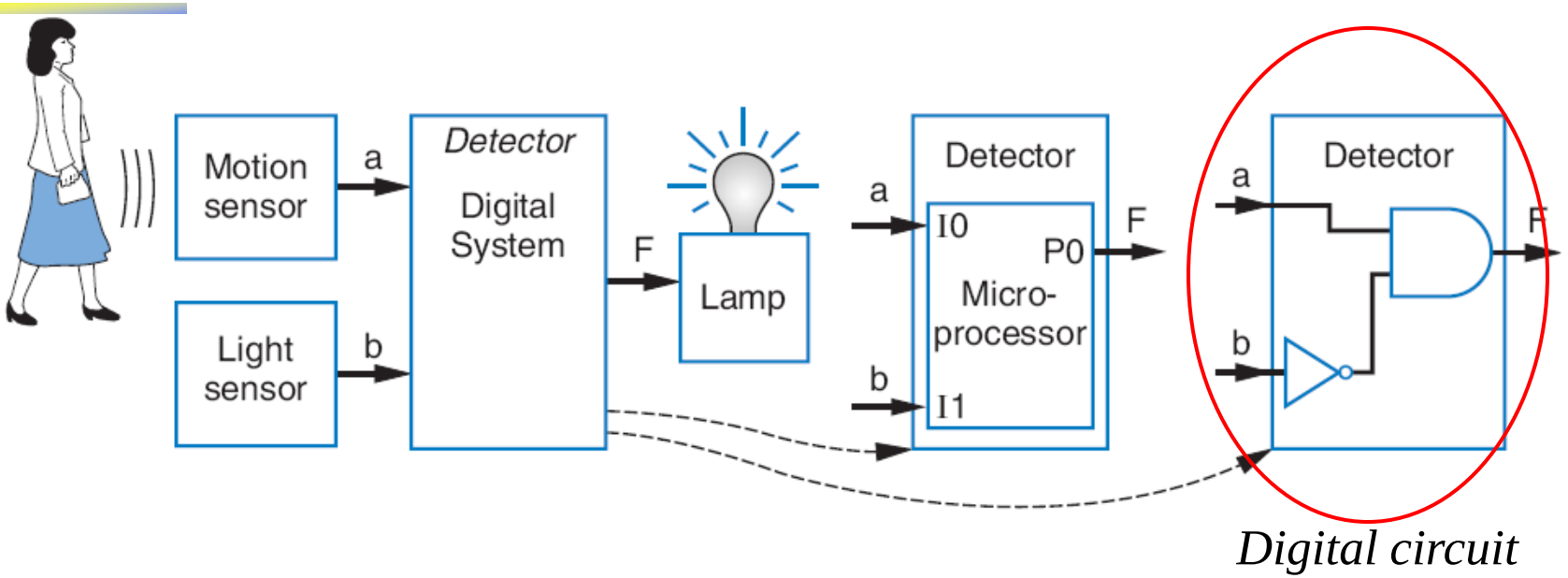


Digital Design

Chapter 2:
Combinational Logic Design

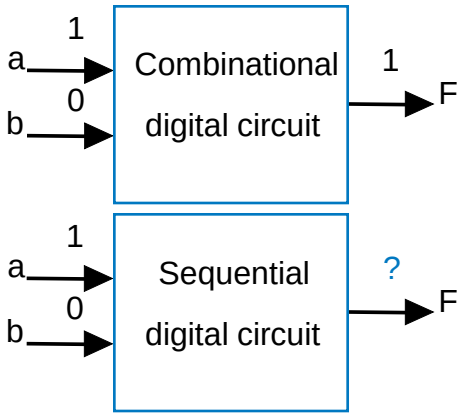
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
http://www.ddvahid.com

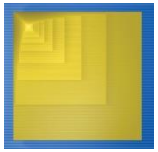
Introduction



Digital circuit

- Let's learn to design digital circuits
- We'll start with a simple form of circuit:
  - ***Combinational circuit***
    - A digital circuit whose outputs depend solely on the *present **combination** of the circuit inputs' values*

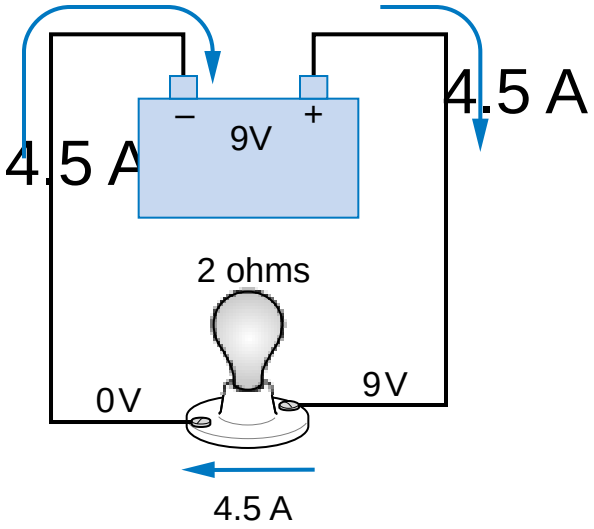


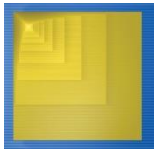


Note: Slides with animation are denoted with a small red "a" near the animated items

Switches

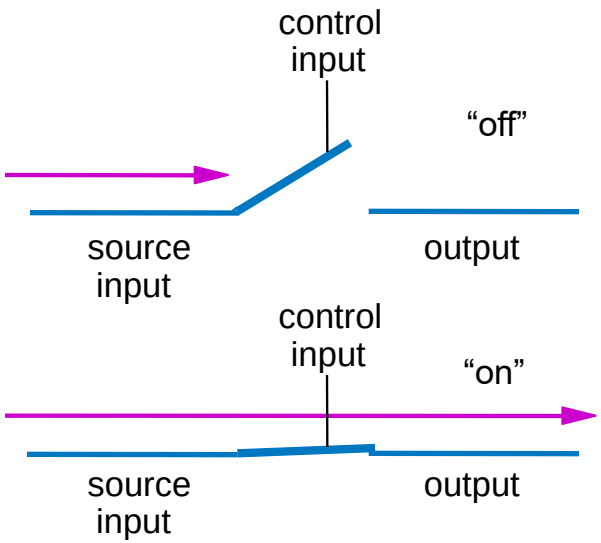
- Electronic switches are the basis of binary digital circuits
  - Electrical terminology
    - ***Voltage***: Difference in electric potential between two points
      - Analogous to water pressure
    - ***Current***: Flow of charged particles
      - Analogous to water flow
    - ***Resistance***: Tendency of wire to resist current flow
      - Analogous to water pipe diameter
    - V = I * R (Ohm's Law)

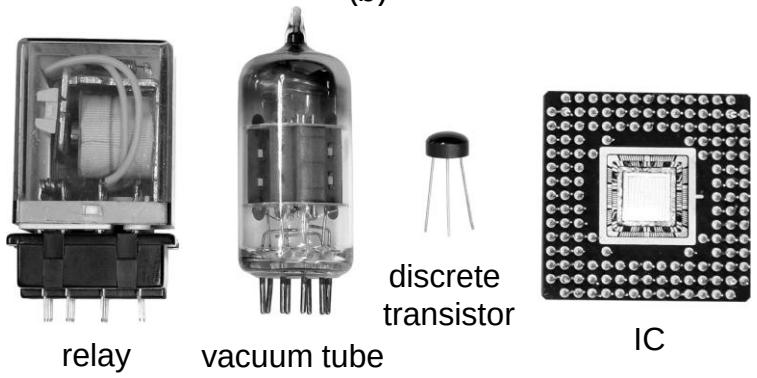




Switches

- A switch has three parts
  - Source input, and output
    - Current wants to flow from source input to output
  - Control input
    - Voltage that controls whether that current can flow
- The amazing shrinking switch
  - 1930s: Relays
  - 1940s: Vacuum tubes
  - 1950s: Discrete transistor
  - 1960s: Integrated circuits (ICs)
    - Initially just a few transistors on IC
    - Then tens, hundreds, thousands...

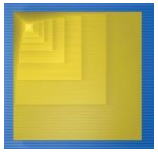




quarter
(to see the relative size)

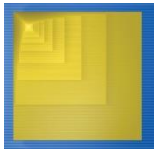
Brief History

- Vacuum Tubes
- 1947-Point Contact Transistor
- 1948-Bipolar Junction Transistor
- 1958-BJT based Integrated Circuit (Prototype)
- pMOSFET
- 1970s-nMOSFET
- 1980s-CMOS Technology
- 2011-FinFET



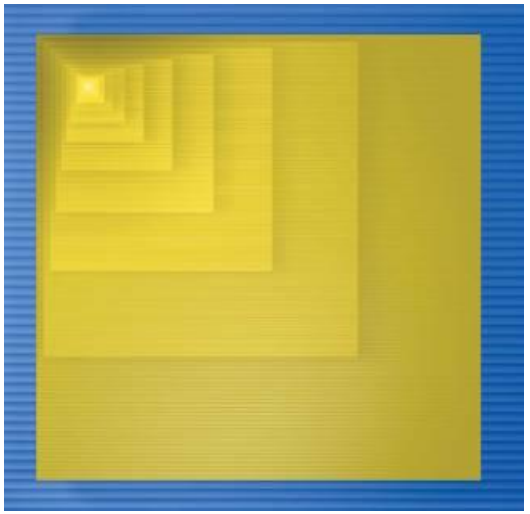
Processor	Transistor count	Date of introduction	Manufacturer	Process	Area
Intel 4004	2,300	1971	Intel	10 µm	12 mm²
Intel 8008	3,500	1972	Intel	10 µm	14 mm²
MOS Technology 6502	3,510	1975	MOS Technology	8 µm	21 mm²
Motorola 6800	4,100	1974	Motorola	6 µm	16 mm²
Intel 8080	4,500	1974	Intel	6 µm	20 mm²
RCA 1802	5,000	1974	RCA	5 µm	27 mm²
Intel 8085	6,500	1976	Intel	3 µm	20 mm²
Zilog Z80	8,500	1976	Zilog	4 µm	18 mm²
Motorola 6809	9,000	1978	Motorola	5 µm	21 mm²
Intel 8086	29,000	1978	Intel	3 µm	33 mm²
Intel 8088	29,000	1979	Intel	3 µm	33 mm²
Intel 80186	55,000	1982	Intel	3 µm	
Motorola 68000	68,000	1979	Motorola	4 µm	44 mm²
Intel 80286	134,000	1982	Intel	1.5 µm	49 mm²
Intel 80386	275,000	1985	Intel	1.5 µm	104 mm²
ARM 1	25,000	1985	Acorn		
ARM 2	25,000	1986	Acorn		
Intel 80486	1,180,235	1989	Intel	1 µm	173 mm²
ARM 3	300,000	1989	Acorn		
R4000	1,350,000	1991	MIPS	1.0 µm	213 mm²
ARM 6	30,000	1991	ARM		
Pentium	3,100,000	1993	Intel	0.8 µm	294 mm²
ARM 7	578977	1994	ARM		68.51 mm²
Pentium Pro	5,500,000	1995	Intel	0.5 µm	307 mm²
AMD K5	4,300,000	1996	AMD	0.5 µm	251 mm²
Pentium II	7,500,000	1997	Intel	0.35 µm	195 mm²
AMD K6	8,800,000	1997	AMD	0.35 µm	162 mm²
Pentium III	9,500,000	1999	Intel	0.25 µm	128 mm²
AMD K6-III	21,300,000	1999	AMD	0.25 µm	118 mm²
AMD K7	22,000,000	1999	AMD	0.25 µm	184 mm²
Pentium 4	42,000,000	2000	Intel	180 nm	217 mm²
Atom	47,000,000	2008	Intel	45 nm	24 mm²
Barton	54,300,000	2003	AMD	130 nm	101 mm²

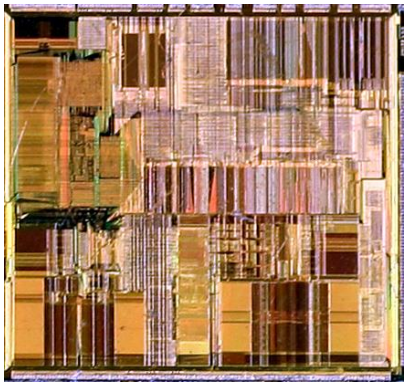
Processor	Transistor count	Date of introduction	Manufacturer	Process	Area
AMD K8	105,900,000	2003	AMD	130 nm	193 mm²
Itanium 2 McKinley	220,000,000	2002	Intel	180 nm	421 mm²
Cell	241,000,000	2006	Sony/IBM/Toshiba	90 nm	221 mm²
Core 2 Duo	291,000,000	2006	Intel	65 nm	143 mm²
Itanium 2 Madison 6M	410,000,000	2003	Intel	130 nm	374 mm²
AMD K10 quad-core 2M L3	463,000,000	2007	AMD	65 nm	283 mm²
ARM Cortex-A9	26,000,000	2007	ARM		
AMD K10 quad-core 6M L3	758,000,000	2008	AMD	45 nm	258 mm²
Itanium 2 with 9MB cache	592,000,000	2004	Intel	130 nm	432 mm²
Core i7	731,000,000	2008	Intel	45 nm	263 mm²
POWER6	789,000,000	2007	IBM	65 nm	341 mm²
Six-Core Opteron 2400	904,000,000	2009	AMD	45 nm	346 mm²
16-Core SPARC T3	1,000,000,000	2010	Sun/Oracle	40 nm	377 mm²
Quad-Core + GPU Core i7	1,160,000,000	2011	Intel	32 nm	216 mm²
Six-Core Core i7	1,170,000,000	2010	Intel	32 nm	240 mm²
8-core POWER7 32M L3	1,200,000,000	2010	IBM	45 nm	567 mm²
8-Core AMD Bulldozer	1,200,000,000	2012	AMD	32nm	315 mm²
Quad-Core + GPU AMD Trinity	1,303,000,000	2012	AMD	32 nm	246 mm²
Quad-core z196	1,400,000,000	2010	IBM	45 nm	512 mm²
Quad-Core + GPU Core i7	1,400,000,000	2012	Intel	22 nm	160 mm²
Dual-Core Itanium 2	1,700,000,000	2006	Intel	90 nm	596 mm²
Six-Core Xeon 7400	1,900,000,000	2008	Intel	45 nm	503 mm²
Quad-Core Itanium Tukwila	2,000,000,000	2010	Intel	65 nm	699 mm²
8-core POWER7+ 80M L3	2,100,000,000	2012	IBM	32 nm	567 mm²
Six-Core Core i7/8-Core Xeon E5	2,270,000,000	2011	Intel	32 nm	434 mm²
8-Core Xeon NehalemEX	2,300,000,000	2010	Intel	45 nm	684 mm²
10-Core Xeon Westmere-EX	2,600,000,000	2011	Intel	32 nm	512 mm²
Six-core zEC12	2,750,000,000	2012	IBM	32 nm	597 mm²
8-Core Itanium Poulson	3,100,000,000	2012	Intel	32 nm	544 mm²
62-Core Xeon Phi	5,000,000,000	2012	Intel	22 nm	
Xbox One Main SoC	5,000,000,000	2013	Microsoft		363 mm²



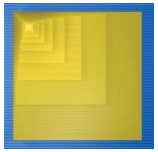
Moore's Law

- IC capacity doubling about every 18 months for several decades
  - Known as "Moore's Law" after Gordon Moore, co-founder of Intel
    - Predicted in 1965 predicted that components per IC would double roughly every year or so
  - Book cover depicts related phenomena
    - For a particular number of transistors, the IC shrinks by half every 18 months
      - Notice how much shrinking occurs in just about 10 years
      - Enables incredibly powerful computation in incredibly tiny devices
  - Today's ICs hold *billions* of transistors
    - The first Pentium processor (early 1990s) needed only 3 million



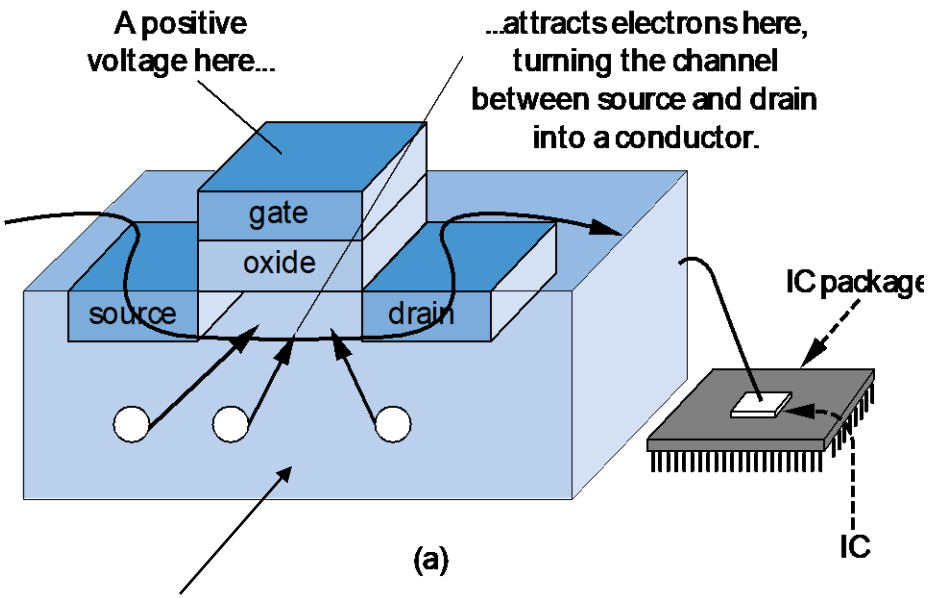


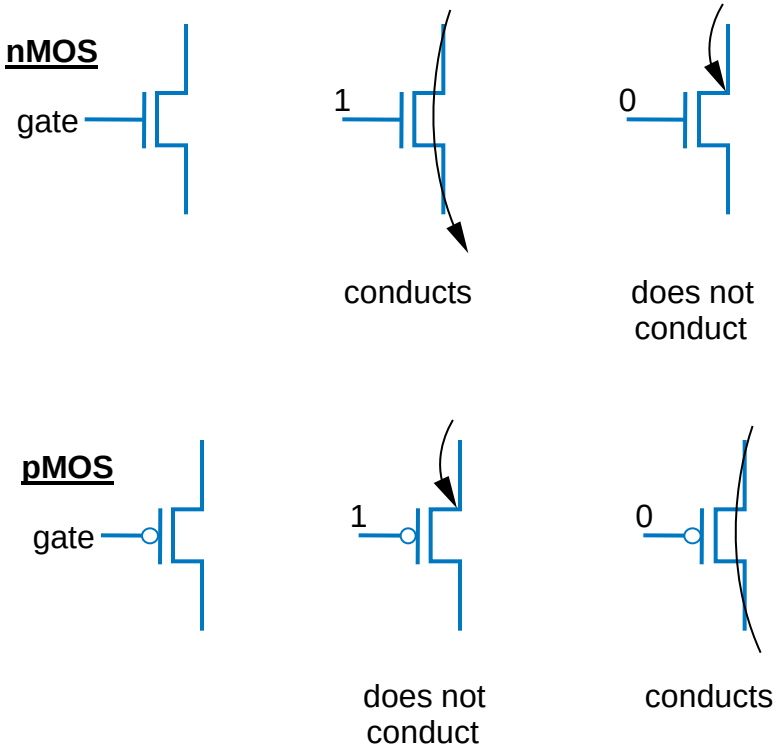
An Intel Pentium processor IC having millions of transistors



The CMOS Transistor

- CMOS transistor
  - Basic switch in modern ICs



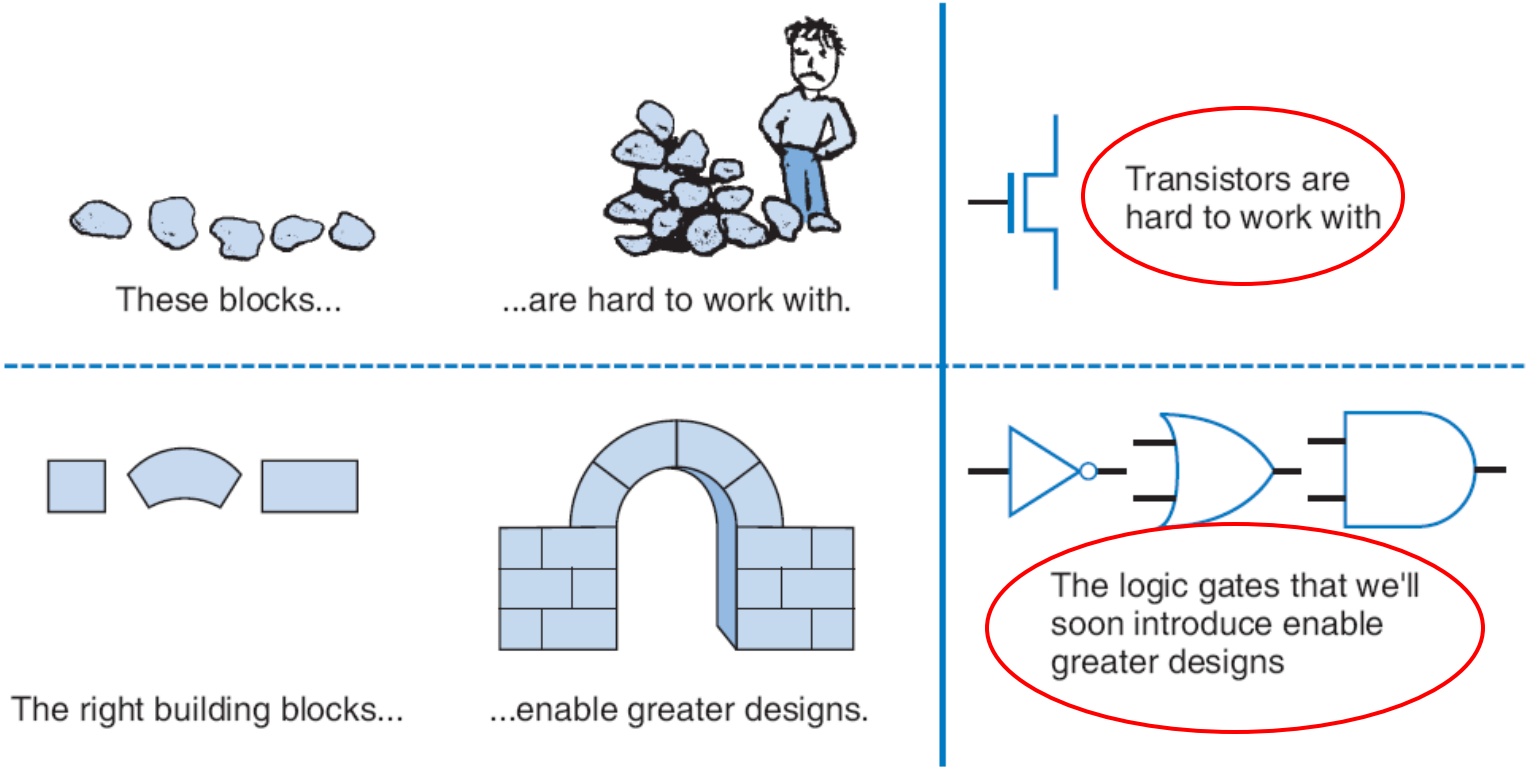


Silicon -- not quite a conductor or insulator:
Semiconductor

Boolean Logic Gates

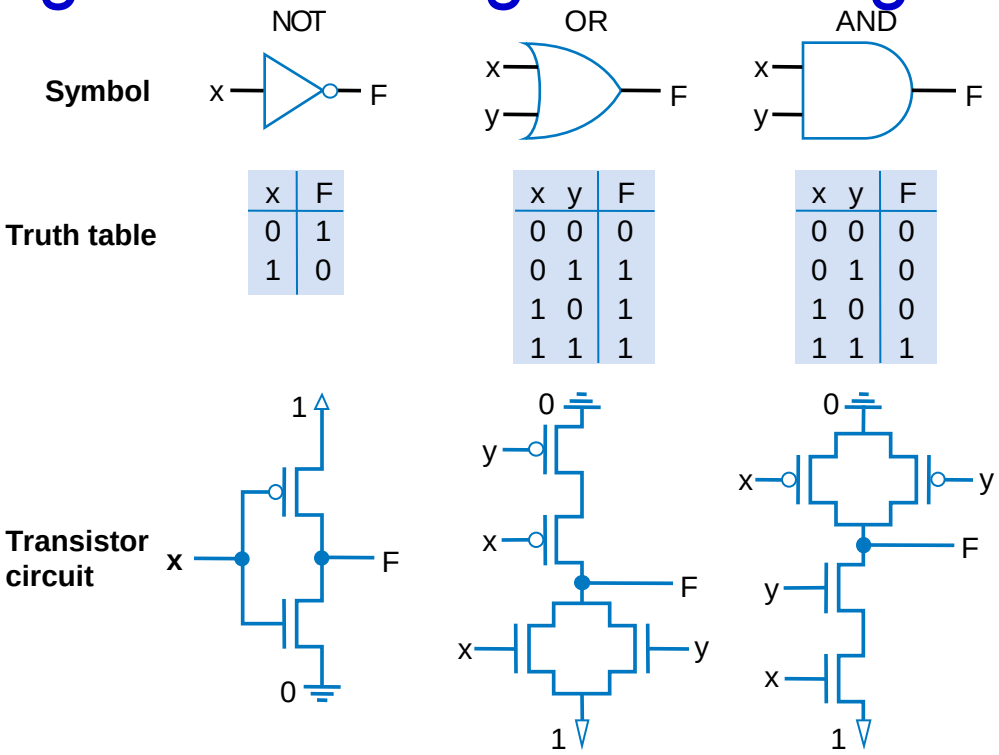
Building Blocks for Digital Circuits

(Because Switches are Hard to Work With)



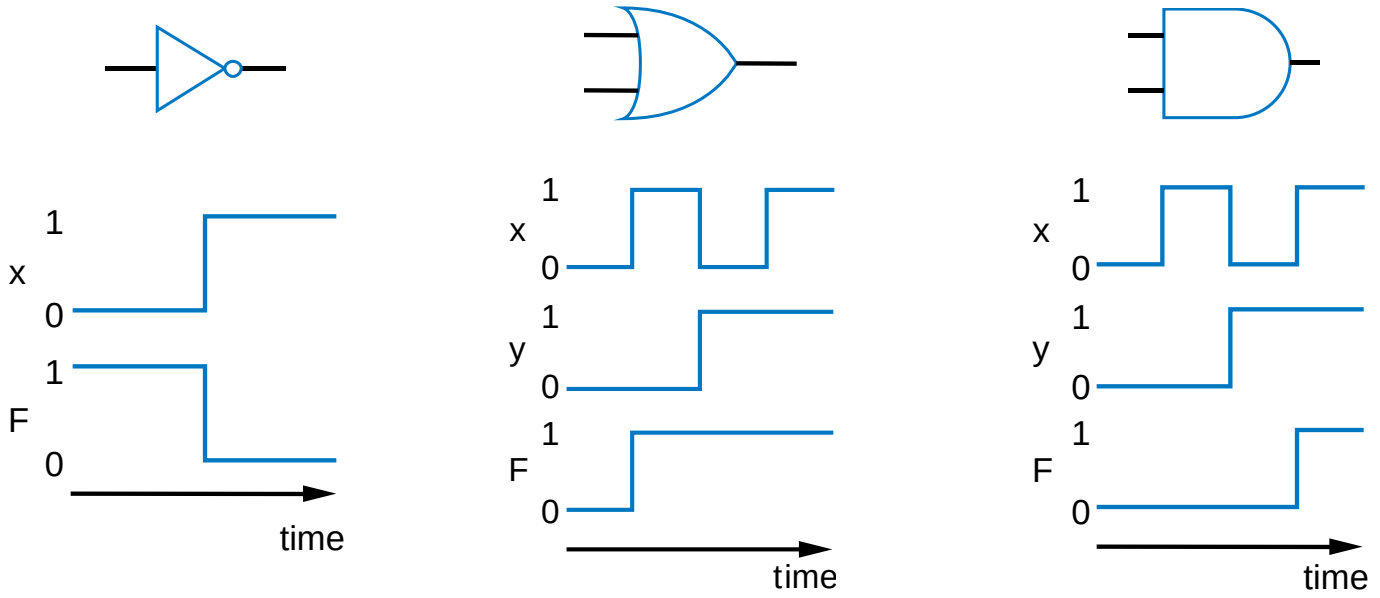
- "Logic gates" are better digital circuit building blocks than switches (transistors)
  - Why?...

Relating Boolean Algebra to Digital Design



- Implement Boolean operators using transistors
  - Call those implementations ***logic gates***.

NOT/OR/AND Logic Gate Timing Diagrams



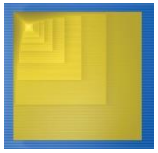
Boolean Algebra and its Relation to Digital Circuits

- ***Boolean Algebra***
  - Variables represent 0 or 1 only
  - Operators return 0 or 1 only
  - Basic operators
    - AND: *a AND b* returns 1 only when both a=1 and b=1
    - OR: *a OR b* returns 1 if either (or both) a=1 or b=1
    - NOT: *NOT a* returns the opposite of a (1 if a=0, 0 if a=1)

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0



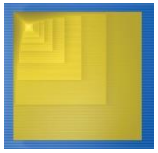
Evaluating Boolean Equations

- Evaluate the Boolean equation **F = (a AND b) OR (c AND d)** for the given values of variables a, b, c, and d:
  - Q1: a=1, b=1, c=1, d=0.
    - Answer: F = (1 AND 1) OR (1 AND 0) = 1 OR 0 = 1.

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

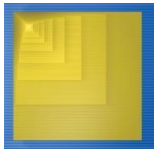
a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0



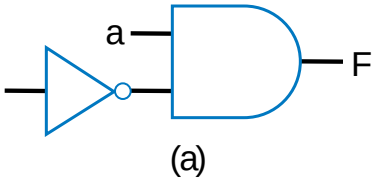
Converting to Boolean Equations

- Convert the following English statements to a Boolean equation
  - Q1. a is 1 and b is 1.
    - Answer: F = a AND b
  - Q2. either of a or b is 1.
    - Answer: F = a OR b
  - Q3. both a and b are not 0.
    - Answer:
      - (a) Option 1: F = NOT(a) AND NOT(b)
      - (b) Option 2: F = a OR b
  - Q4. a is 1 and b is 0.
    - Answer: F = a AND NOT(b)



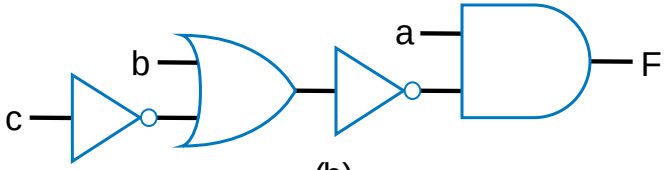
Example: Converting a Boolean Equation to a Circuit of Logic Gates

- Q: Convert the following equation to logic gates:
  F = a AND NOT(b OR NOT(c))



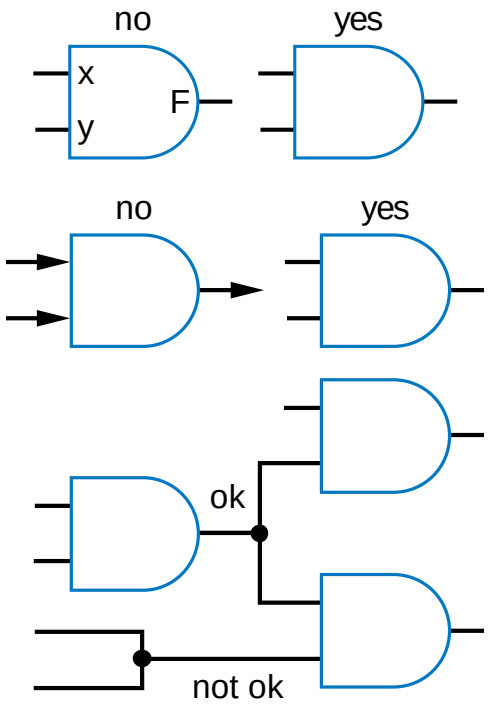
(a)

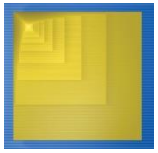
a



(b)

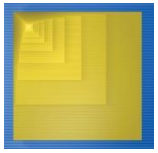
Some Circuit Drawing Conventions





Boolean Algebra Terminology

- Example equation: **F(a,b,c) = a'bc + abc' + ab + c**
- ***Variable***
  - Represents a value (0 or 1)
  - Three variables: a, b, and c
- ***Literal***
  - Appearance of a variable, in true or complemented form
  - Nine literals: a', b, c, a, b, c', a, b, and c
- ***Product term***
  - Product of literals
  - Four product terms: a'bc, abc', ab, c
- ***Sum-of-products***
  - Equation written as OR of product terms only
  - Above equation is in sum-of-products form. "F = (a+b)c + d" is not.

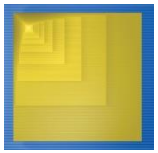


Boolean Algebra Properties

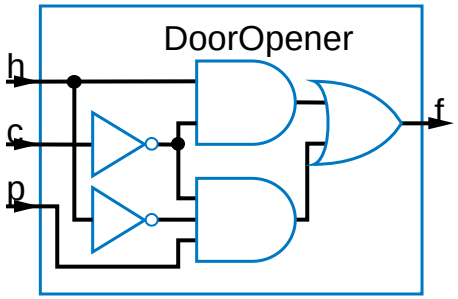
- Commutative
  - a + b = b + a
  - a * b = b * a
- Distributive
  - a * (b + c) = a * b + a * c
  - a + (b * c) = (a + b) * (a + c)
    - (this one is tricky!)
- Associative
  - (a + b) + c = a + (b + c)
  - (a * b) * c = a * (b * c)
- Identity
  - 0 + a = a + 0 = a
  - 1 * a = a * 1 = a
- Complement
  - a + a' = 1
  - a * a' = 0
- To prove, just evaluate all possibilities

Example uses of the properties

- Show abc' equivalent to c'ba.
  - Use commutative property:
    - a*b*c' = a*c'*b = c'*a*b = c'*b*a = c'ba.
- Show abc + abc' = ab.
  - Use first distributive property
    - abc + abc' = ab(c+c').
  - Complement property
    - Replace c+c' by 1: ab(c+c') = ab(1).
  - Identity property
    - ab(1) = ab*1 = ab.
- Show x + x'z equivalent to x + z.
  - Second distributive property
    - Replace x+x'z by (x+x')*(x+z).
  - Complement property
    - Replace (x+x') by 1,
  - Identity property
    - replace 1*(x+z) by x+z.



Example that Applies Boolean Algebra Properties



- Found inexpensive chip that computes:
  - f = c'hp + c'hp' + c'h'p
  - Can we use it?
    - Is it the same as f = c'(p+h)?
- Use Boolean algebra:

f = c'hp + c'hp' + c'h'p

f = c'h(p + p') + c'h'p (by the distributive property)

f = c'h(1) + c'h'p (by the complement property)

f = c'h + c'h'p (by the identity property)

f = hc' + h'pc' (by the commutative property)

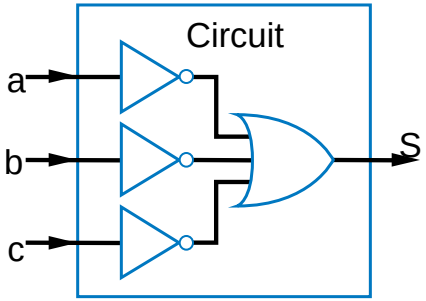
Same!

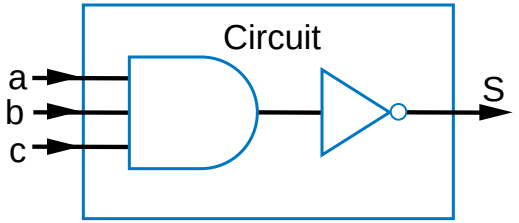
Boolean Algebra: Additional Properties

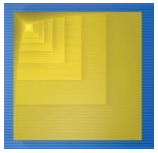
- Null elements
  - a + 1 = 1
  - a * 0 = 0
- Idempotent Law
  - a + a = a
  - a * a = a
- Involution Law
  - (a')' = a
- DeMorgan's Law
  - (a + b)' = a'b'
  - (ab)' = a' + b'
  - Very useful!
- To prove, just evaluate all possibilities

Aircraft lavatory sign example

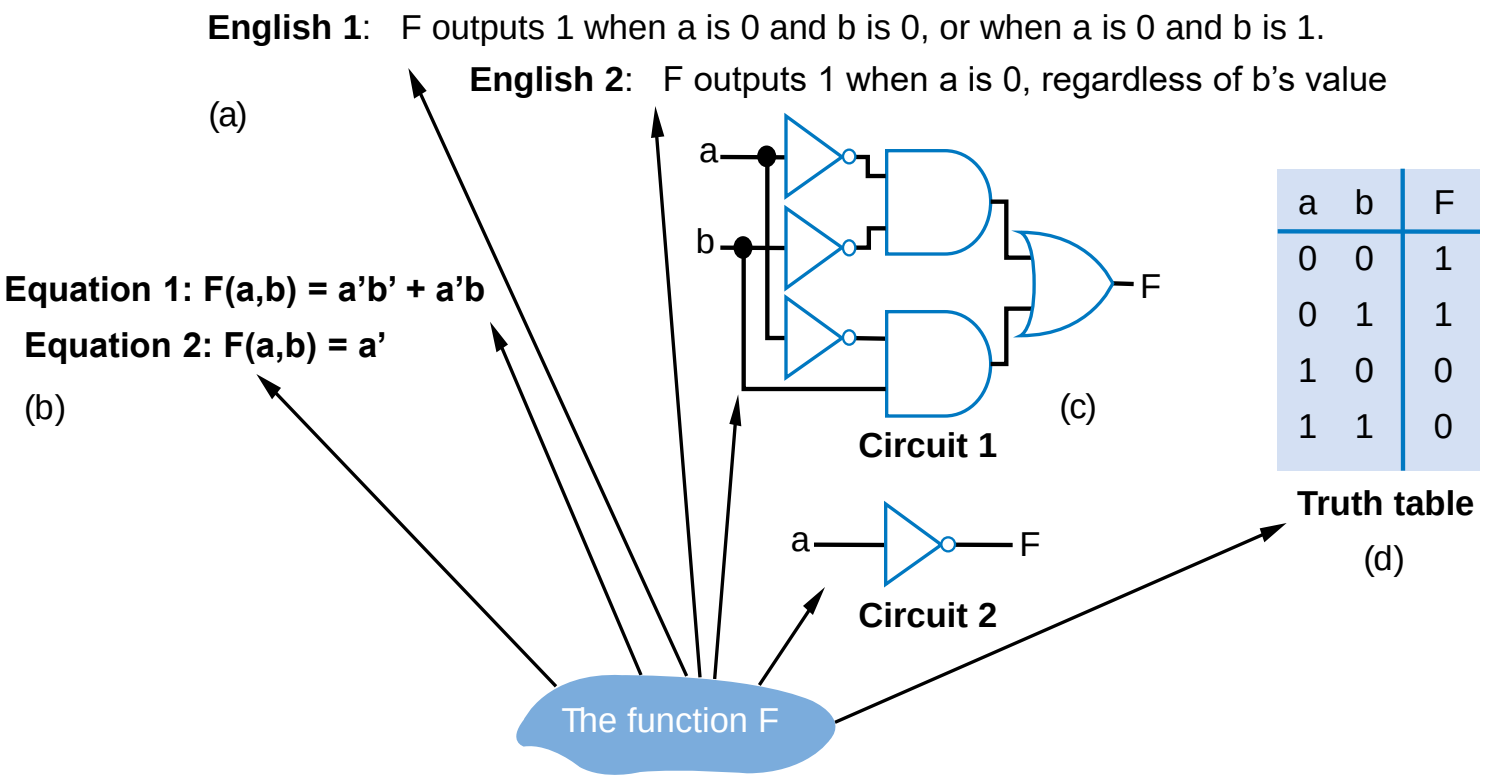
- Equation and circuit
  - S = a' + b' + c'
- Transform
  - (abc)' = a'+b'+c' (by DeMorgan's Law)
  - S = (abc)'
- New equation and circuit







Representations of Boolean Functions



- A function can be represented in different ways
  - Above shows seven representations of the same functions F(a,b), using four different methods: English, Equation, Circuit, and Truth Table

Truth Table Representation of Boolean Functions

- Define value of F for each possible combination of input values
  - 2-input function: 4 rows
  - 3-input function: 8 rows
  - 4-input function: 16 rows
- Q: Use truth table to define function F(a,b,c) that is 1 when abc is 5 or greater in binary

a	b	F
0	0	
0	1	
1	0	
1	1	

(a)

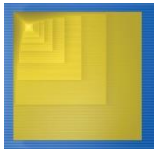
a	b	c	F
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(b)

a	b	c	d	F
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

(c)

a	b	c	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**
0	1	1	**0**
1	0	0	**0**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**



Converting among Representations

- Can convert from any representation to any other
- Common conversions
  - Equation to circuit
  - Truth table to equation
  - Equation to truth table
    - Easy -- just evaluate equation for each input combination (row)
    - Creating intermediate columns helps

Inputs		Outputs	Term
a	b	F	F = sum of
0	0	1	a'b'
0	1	1	a'b
1	0	0	
1	1	0	

F = a'b' + a'b

Q: Convert to equation

a	b	c	F	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	ab'c
1	1	0	1	abc'
1	1	1	1	abc

F = ab'c + abc' + abc

Q: Convert to truth table: F = a'b' + a'b

Inputs				Output
a	b	a' b'	a' b	F
0	0	1	0	**1**
0	1	0	1	**1**
1	0	0	0	**0**
1	1	0	0	**0**

Standard Representation: Truth Table

- How can we determine if two functions are the same?
  - Use algebraic methods
  - But if we failed, does that prove *not* equal? No.
- Solution: Convert to truth tables
  - Only ONE truth table representation of a given function
    - ***Standard*** representation -- for given function, only one version in standard form exists

f = c'hp + c'hp' + c'h'

f = c'h(p + p') + c'h'p

f = c'h(1) + c'h'p

f = c'h + c'h'p

(what if we stopped here?)

f = hc' + h'pc'

F = ab + 'a

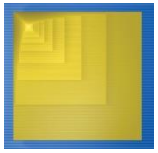
a	b	F
0	0	1
0	1	1
1	0	0
1	1	1

Same

F = a'b' + a'b + ab

a	b	F
0	0	1
0	1	1
1	0	0
1	1	1

a



Canonical Form -- Sum of Minterms

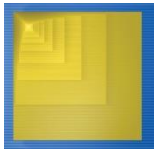
- Truth tables too big for numerous inputs
- Use standard form of equation instead
  - Known as ***canonical form***
  - Regular algebra: group terms of polynomial by power
    - $ax^2 + bx + c$ $(3x^2 + 4x + 2x^2 + 3 + 1 \text{ --> } 5x^2 + 4x + 4)$
  - Boolean algebra: create sum of minterms
    - ***Minterm***: product term with every function literal appearing exactly once, in true or complemented form
    - Just multiply-out equation until sum of product terms
    - Then expand each term until all terms are minterms

Q: Determine if F(a,b)=ab+a' is same function as F(a,b)=a'b'+a'b+ab, by converting first equation to canonical form (second already in canonical form)

F = ab+a' (already sum of products)

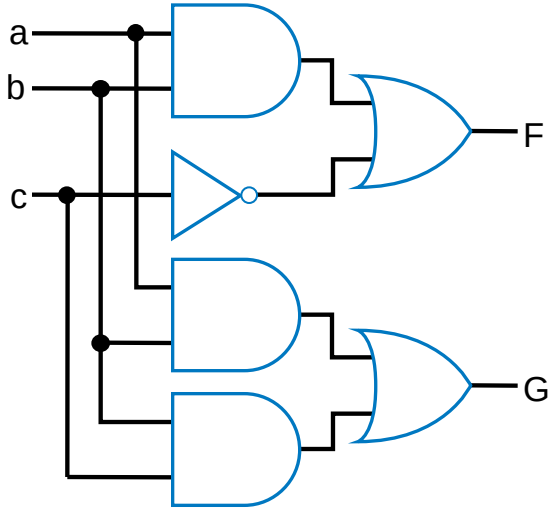
F = ab + a'(b+b') (expanding term)

F = ab + a'b + a'b' (SAME -- same three terms as other equation)



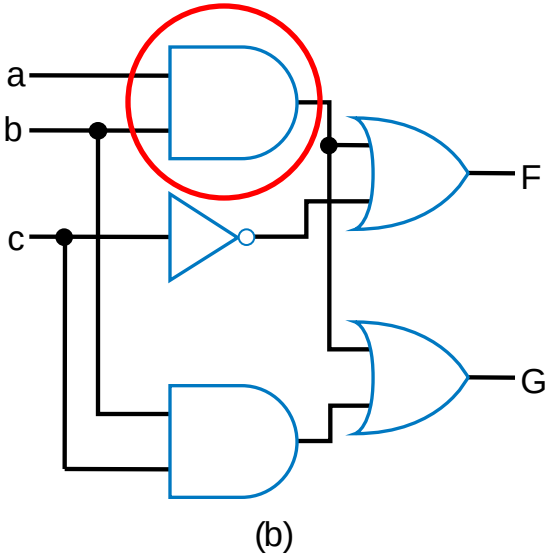
Multiple-Output Circuits

- Many circuits have more than one output
- Can give each a separate circuit, or can share gates
- Ex: F = ab + c', G = ab + bc



(a)

Option 1: Separate circuits



(b)

Option 2: Shared gates

Multiple-Output Example: BCD to 7-Segment Converter

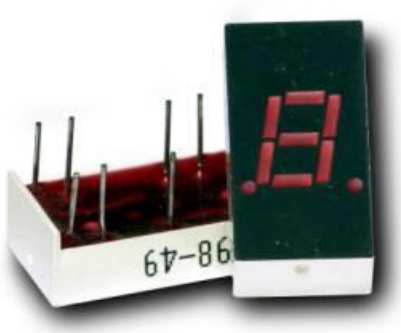
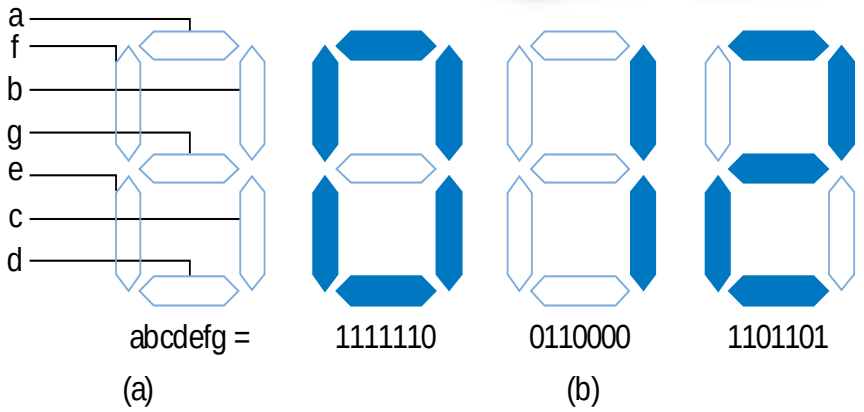


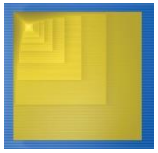
TABLE 2-4 4-bit binary number to seven-segment display truth table

w	x	y	z	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



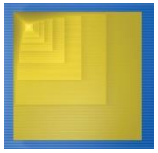
a = w'x'y'z' + w'x'yz' + w'x'yz + w'xy'z + w'xyz' + w'xyz + wx'y'z' + wx'y'z

b = w'x'y'z' + w'x'y'z + w'x'yz' + w'x'yz + w'xy'z' + w'xyz + wx'y'z' + wx'y'z



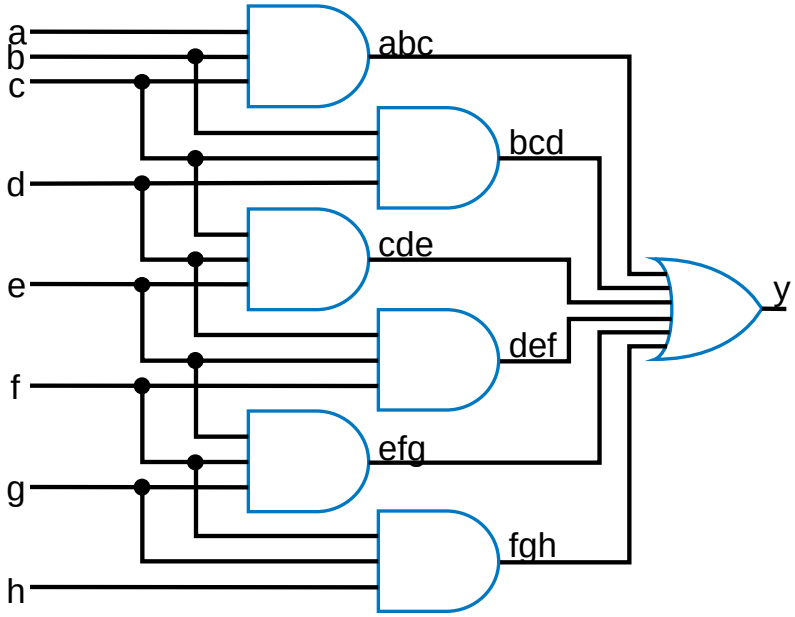
Combinational Logic Design Process

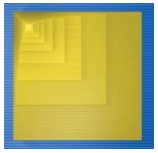
	Step	Description
Step 1	**Capture** the function	Create a truth table or equations, ***whichever is most natural for the given problem***, to describe the desired behavior of the combinational logic.
Step 2	**Convert** to equations	This step is only necessary if you captured the function using a truth table instead of equations. Create an equation for each output by ORing all the minterms for that output. Simplify the equations if desired.
Step 3	**Implement** as a gate-based circuit	For each output, create a circuit corresponding to the output's equation. (Sharing gates among multiple outputs is OK optionally.)



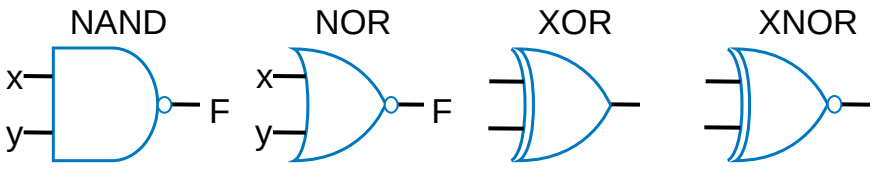
Example: Three 1s Detector

- Problem: Detect three consecutive 1s in 8-bit input: abcdefgh
  - 000**111**01 → 1 10101011 → 0
    11110000 → 1
  - **Step 1: Capture** the function
    - Truth table or equation?
      - Truth table too big: 2^8=256 rows
      - Equation: create terms for each possible case of three consecutive 1s
    - **y = abc + bcd + cde + def + efg + fgh**
  - **Step 2: Convert** to equation -- already done
  - **Step 3: Implement** as a gate-based circuit





More Gates

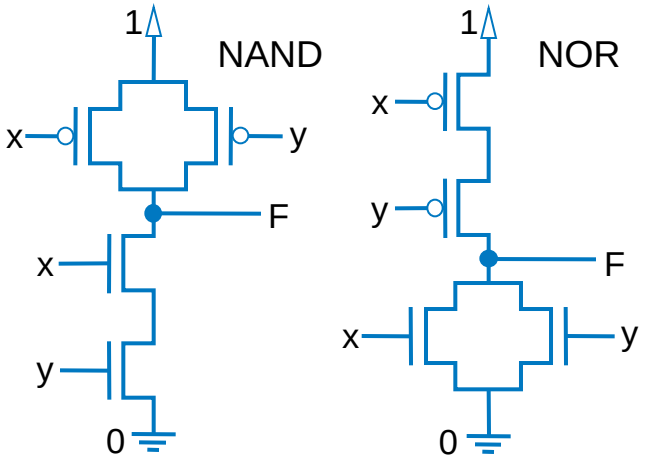


x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

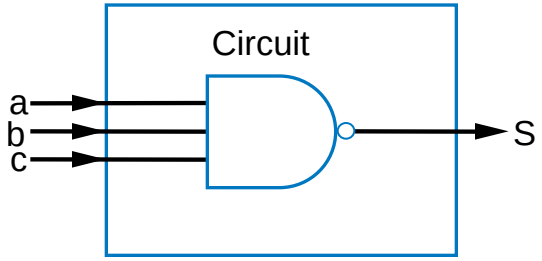


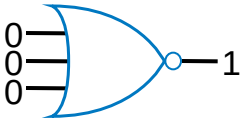
- NAND: Opposite of AND (“NOT AND”)
- NOR: Opposite of OR (“NOT OR”)
- XOR: Exactly 1 input is 1, for 2-input XOR. (For more inputs -- odd number of 1s)
- XNOR: Opposite of XOR (“NOT XOR”)

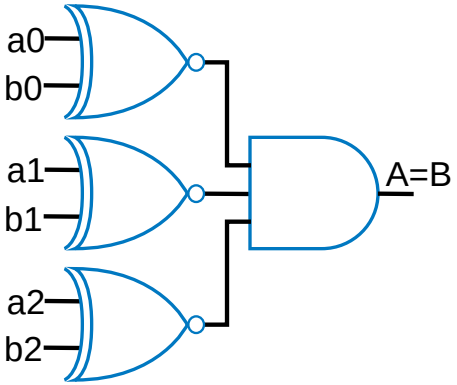
- NAND same as AND with power & ground switched
  - Why? nMOS conducts 0s well, but not 1s (reasons beyond our scope) -- so NAND more efficient
- Likewise, NOR same as OR with power/ground switched
- AND in CMOS: NAND with NOT
- OR in CMOS: NOR with NOT
- So NAND/NOR more common

More Gates: Example Uses

- Aircraft lavatory sign example
  - S = (abc)'
- Detecting all 0s
  - Use NOR
- Detecting equality
  - Use XNOR
- Detecting odd # of 1s
  - Use XOR
  - Useful for generating "parity" bit common for detecting errors

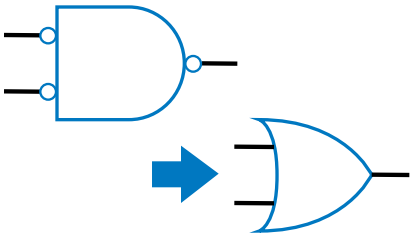


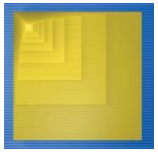




Completeness of NAND

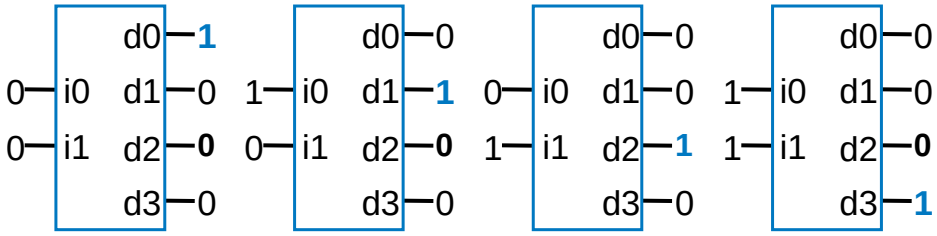
- Any Boolean function can be implemented *using just NAND gates.* Why?
  - Need AND, OR, and NOT
  - NOT: 1-input NAND (or 2-input NAND with inputs tied together)
  - AND: NAND followed by NOT
  - OR: NAND preceded by NOTs
- Likewise for NOR

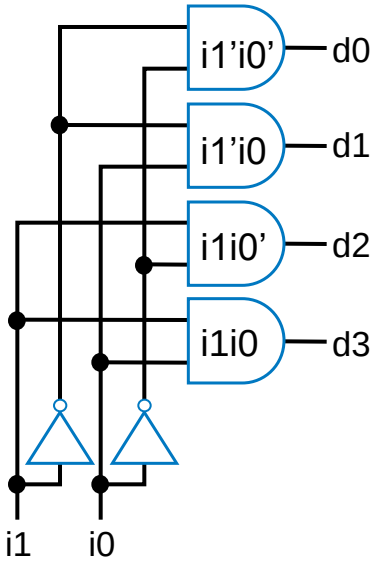


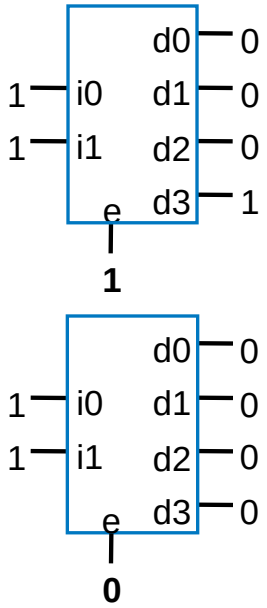


Decoders and Muxes

- **Decoder**: Popular combinational logic building block, in addition to logic gates
  - Converts input binary number to one high output
- 2-input decoder: four possible input binary numbers
  - So has four outputs, one for each possible input binary number
- Internal design
  - AND gate for each output to detect input combination
- Decoder with enable e
  - Outputs all 0 if e=0
  - Regular behavior if e=1
- n-input decoder: 2^n outputs

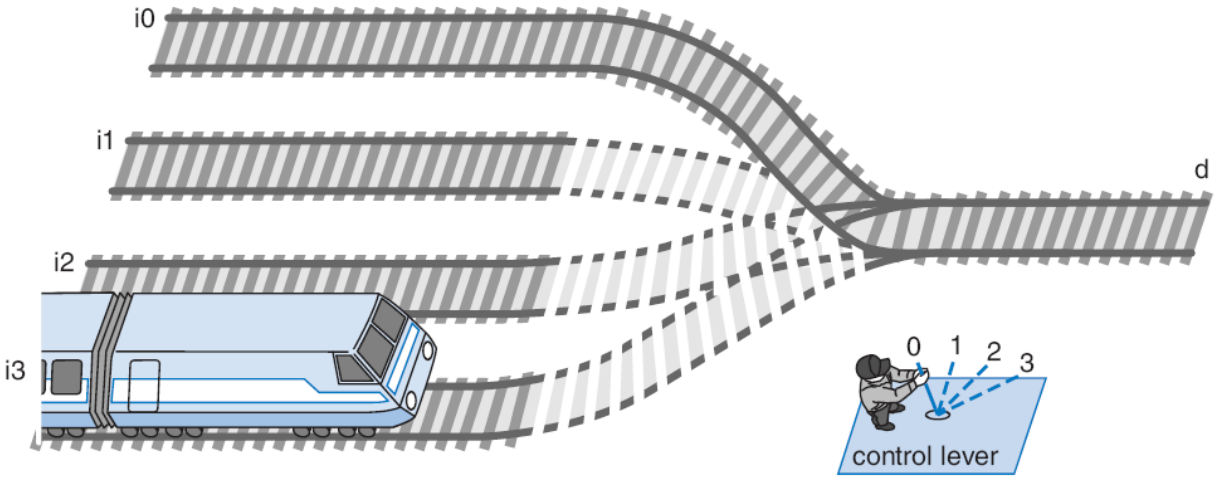




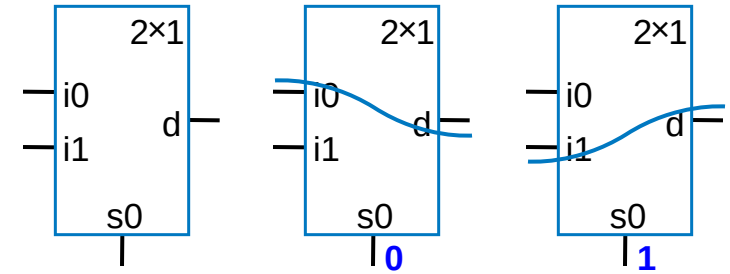


Multiplexor (Mux)

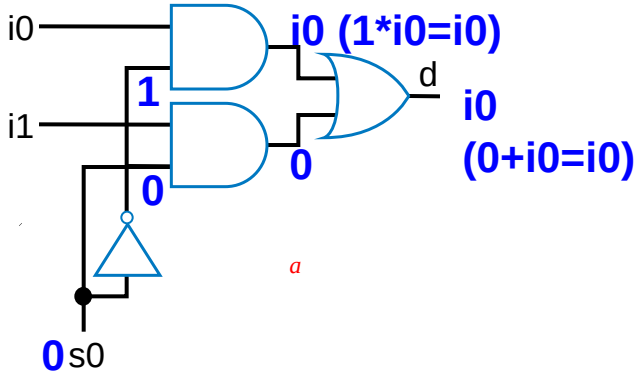
- Mux: Another popular combinational building block
  - Routes one of its N data inputs to its one output, based on binary value of select inputs
    - 4 input mux → needs 2 select inputs to indicate which input to route through
    - 8 input mux → 3 select inputs
    - N inputs → $\log_2(N)$ selects
  - Like a railyard switch

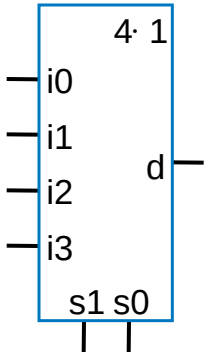


Mux Internal Design

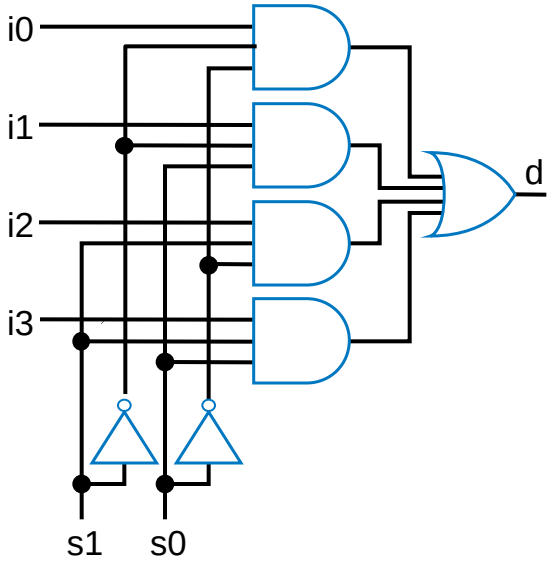


2x1 mux



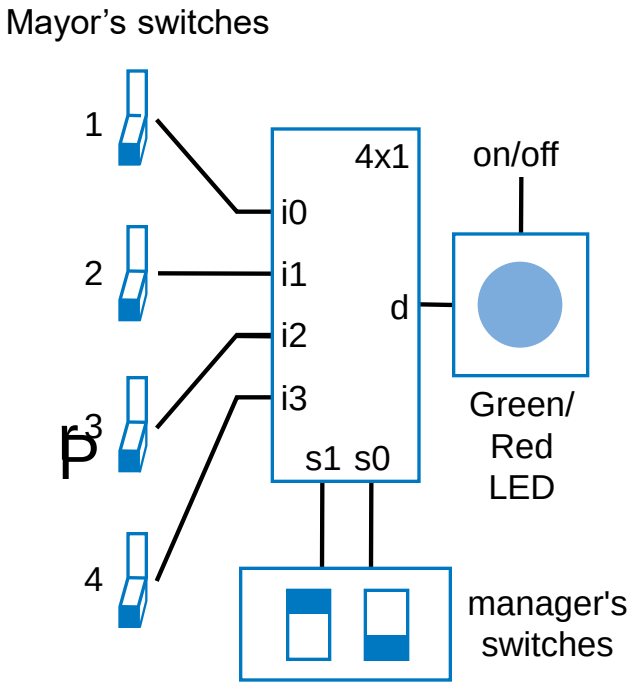


4x1 mux

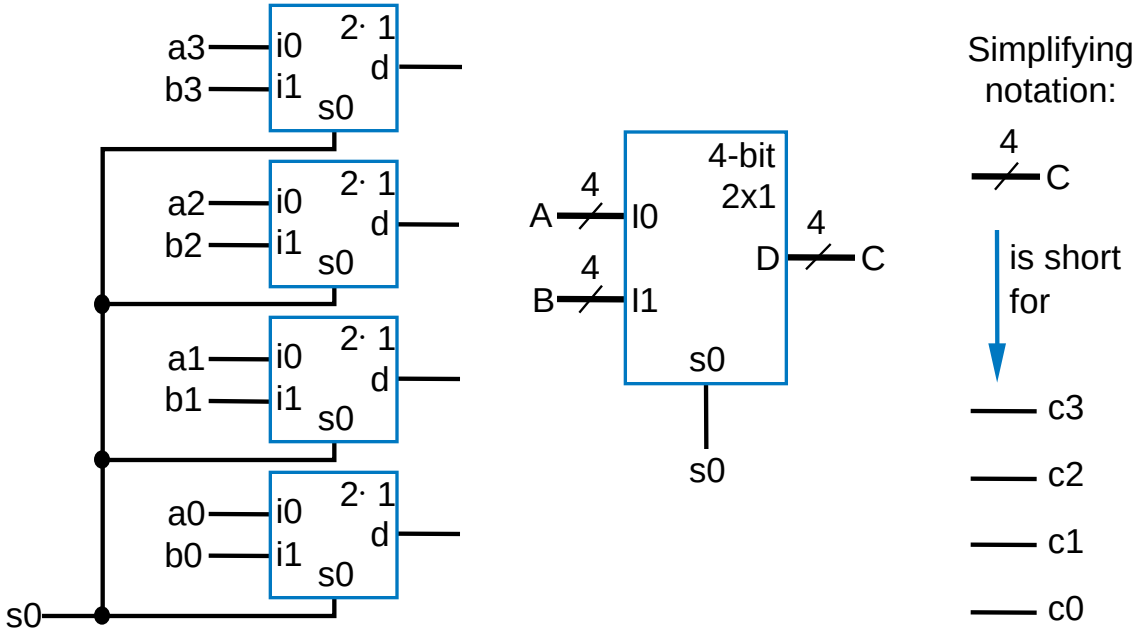


Mux Example

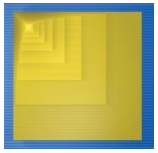
- City mayor can set four switches up or down, representing his/her vote on each of four proposals, numbered 0, 1, 2, 3
- City manager can display any such vote on large green/red LED (light) by setting two switches to represent binary 0, 1, 2, or 3
- Use 4x1 mux



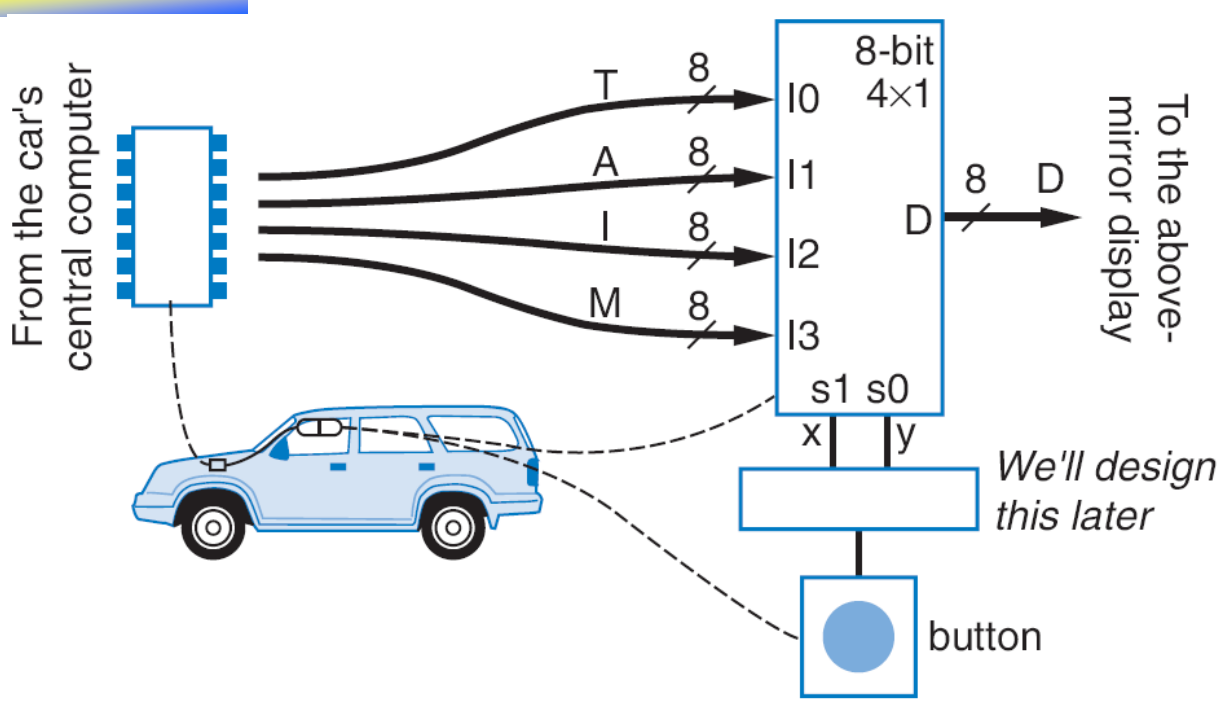
Muxes Commonly Together -- N-bit Mux



- Ex: Two 4-bit inputs, A (a3 a2 a1 a0), and B (b3 b2 b1 b0)
  - 4-bit 2x1 mux (just four 2x1 muxes sharing a select line) can select between A or B



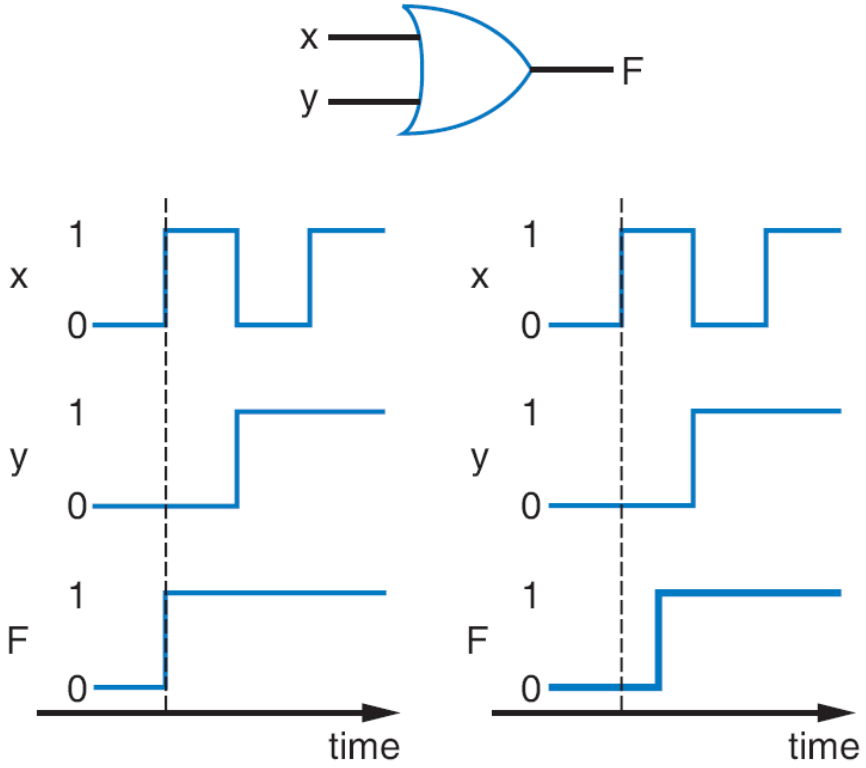
N-bit Mux Example



- Four possible display items
  - Temperature (T), Average miles-per-gallon (A), Instantaneous mpg (I), and Miles remaining (M) -- each is 8-bits wide
  - Choose which to display using two inputs x and y
  - Use 8-bit 4x1 mux

Additional Considerations

Non-Ideal Gate Behavior -- Delay



- Real gates have some delay
  - Outputs don't change immediately after inputs change

Chapter Summary

- Combinational circuits
  - Circuit whose outputs are function of present inputs
    - No “state”
- Switches: Basic component in digital circuits
- Boolean logic gates: AND, OR, NOT -- Better building block than switches
  - Enables use of Boolean algebra to design circuits
- Boolean algebra: uses true/false variables/operators
- Representations of Boolean functions: Can translate among
- Combinational design process: Translate from equation (or table) to circuit through well-defined steps
- More gates: NAND, NOR, XOR, XNOR also useful
- Muxes and decoders: Additional useful combinational building blocks